SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent Demo)
Weightage:	20% of CIA	**Total Marks:**	2 * 50 = 100 (1 Question1)
CO3 Statement:	*Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)*		
Student Name:	Joanathan Packia Singh	**Reg. No.:**	192472229
Section / Batch:	SLOT A	**Date:**	09/08/2026

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Implement the solution using **Python**.
- Use an appropriate LLM such as **OpenAI, Gemini, or Hugging Face**.
- Use **embeddings and semantic search** where applicable.
- Use **FAISS or ChromaDB** as the vector database for RAG tasks.
- Demonstrate the complete pipeline:

Document → Chunking → Embedding → Retrieval → Generation

- Demonstrate the system with multiple queries.
- Handle irrelevant, incomplete, or unsupported queries appropriately.
- Display retrieved context/source wherever applicable.
- Explain the system architecture.
- Provide a working end-to-end demonstration.

Code of Conduct

I Joanathan Packia Singh/192472229 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan

SECTION A – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

21. Banking Policy RAG Assistant: Develop a RAG application using banking policy documents. Answer questions about account opening, loans, services, transaction limits, and procedures.
40. Complete RAG + AI Agent Demonstration: Develop a complete RAG-based AI Agent integrating document loading, chunking, embeddings, semantic search, FAISS/ChromaDB, retrieval, LLM generation, query handling, tool calling, source citation, and unsupported-query handling.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Pipeline Functionality	30%	Correct RAG pipeline — embeddings, retrieval and generation all function coherently	Pipeline mostly correct with minor gaps	Pipeline only partially functional	Pipeline non-functional or conceptually flawed
Answer Relevance & Groundedness	25%	Retrieves relevant context and generates accurate, grounded answers	Mostly relevant/accurate answers	Inconsistent relevance or accuracy	Answers largely irrelevant or hallucinated
Query Robustness	20%	Robust handling of varied queries and documents	Handles most queries well	Handles only simple queries	Fails on basic queries
End-to-End Demo	15%	Smooth, well-demonstrated end-to-end system	Demo mostly smooth, minor hiccups	Demo has noticeable issues	Demo fails or is not ready
Architecture Explanation	10%	Clear architecture diagram and explanation of design decisions	Adequate explanation of design	Minimal explanation of design	No explanation of design provided

Marks Evaluation:

Question No.	Pipeline Functionality(30%)	Answer Relevance & Groundedness (25%)	Query Robustness(20%)	End-to-End Demo(15%)	Architecture Explanation (10%)	Total Marks (100)
Q1						
Q2						
					Overall Total (100)	

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Department of Computer Science and Engineering

Engineer to Excel

RAG-BASED MINI ASSIGNMENT

Banking Policy RAG Assistant & Complete RAG + AI Agent Demonstration

Course Code / Title	CSA65 — Generative AI and Large Language Models
Assessment	Assessment Tool 2 — RAG-based Mini Assignment (Document QA / AI Agent Demo)
CO Assessed	CO3 — Precision (Bloom Levels BL2, BL3)
CO3 Statement	Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.
Student Name	Joanathan Packia Singh
Registration No.	192472229
Section / Batch	SLOT A
Date	09/08/2026
Weightage	20% of CIA • Total Marks: 100 (2 × 50)

Code of Conduct

I, Joanathan Packia Singh (192472229), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan

Table of Contents

1. Introduction and Common Setup

This report answers both questions assigned under Assessment Tool 2 (RAG-based Mini Assignment): a Banking Policy RAG Assistant that answers customer questions from bank policy documents, and a complete RAG + AI Agent demonstration that extends the same pipeline with tool calling, source citation, and explicit unsupported-query handling.

Both solutions are implemented in Python, use an LLM API (Hugging Face / OpenAI-compatible chat completion) for generation, embeddings for semantic search, and FAISS as the vector database, following the required Document → Chunking → Embedding → Retrieval → Generation pipeline. Both systems are demonstrated with multiple queries — including irrelevant, incomplete, and unsupported ones — and every grounded answer displays the source document it was retrieved from.

Common Design Principles Applied to Both Questions

- Modular pipeline functions — chunking, embedding, retrieval, and generation are each independently testable.
- Context-only generation — the LLM prompt explicitly restricts answers to retrieved content, so unsupported queries are declined rather than hallucinated.
- Source transparency — every grounded answer is returned together with the document (and chunk id, in Question 2) it came from.
- Secrets management — API keys are read from environment variables / Colab secrets and never hard-coded.

2. System Architecture

Both questions run on the same underlying system, so the architecture is described once here and referenced from each question's own approach section. The system is a five-layer, single-process Python application — there is no client-server split, database server, or hosted backend; every layer runs inside the same script and the only network calls are stateless HTTPS requests to two external APIs.

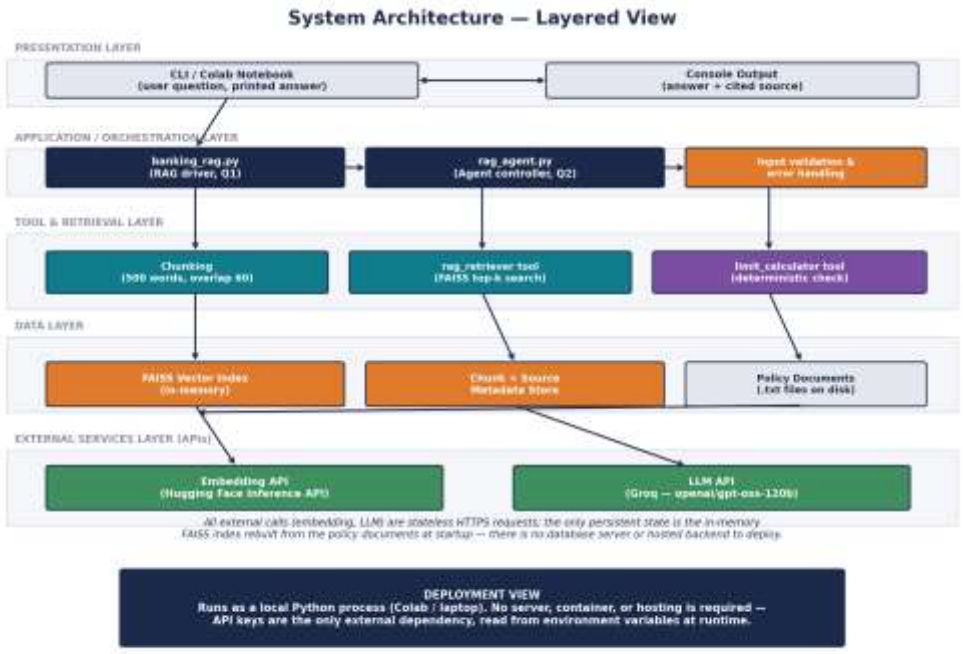


Figure 0. Layered system architecture shared by both the Banking Policy RAG Assistant (Q1) and the RAG + Agent system (Q2).

2.1 Layer Responsibilities

- Presentation Layer — a command-line / Colab-notebook interface that reads the user's question and prints the final answer; this is the only layer the user directly interacts with.
- Application / Orchestration Layer — banking_rag.py and rag_agent.py, which coordinate the pipeline: validate input, decide what to retrieve, and assemble the final prompt. The agent controller in rag_agent.py additionally decides which tool(s) to invoke for a given query.
- Tool & Retrieval Layer — the chunking function, the rag_retriever tool (semantic search), and, in Question 2, the limit_calculator tool (deterministic arithmetic). Each is a stateless function the orchestration layer calls and gets a result back from.
- Data Layer — the FAISS vector index and an in-memory list mapping each chunk back to its source document, both rebuilt from the raw policy .txt files each time the program starts.
- External Services Layer — the two third-party APIs the system depends on: the Hugging Face Inference API for embeddings, and the Groq LLM API for answer generation. Both are called over HTTPS and neither is self-hosted.

2.2 Data Flow

A request flows strictly downward through the layers and the response flows back up: the user's question enters at the Presentation layer, is validated and routed by the Orchestration layer, triggers one or more calls in the Tool layer, which read from the Data layer and, where needed, call out to the External Services layer for embeddings or generation — the response is then passed back up through the same layers to the user. No layer is skipped, which keeps every request traceable and makes it easy to see exactly which component would need to change if a requirement changed (for example, swapping FAISS for ChromaDB only touches the Data layer).

2.3 Deployment View

Because there is no persistent server component, the system deploys as a single Python process — a Colab notebook cell or a local script invocation. The FAISS index is an in-memory artifact rebuilt at startup rather than a managed database, and the two API keys (HF_API_TOKEN, GROQ_API_KEY) are the system's only external dependencies. This keeps the assignment's scope appropriately limited to the RAG pipeline itself: there is intentionally no web server, container, or hosting configuration to build, since none is required by the brief.

This also means the architecture scales horizontally in the trivial sense that any number of independent copies of the process can run in parallel (e.g., one per user session) without needing to coordinate shared state, since each copy rebuilds its own index from the same source documents.

3. Question 1 — Banking Policy RAG Assistant

3.1 Problem Statement

Develop a RAG application using banking policy documents that answers customer questions about account opening, loans, services, transaction limits, and procedures (Question 21).

3.2 Approach and Architecture

Bank policy documents are chunked, embedded, and stored in a FAISS index offline. At query time the customer's question is embedded and matched against the index; the top-k retrieved chunks (each tagged with its source

document) are inserted into a context-only prompt sent to the LLM, which must answer strictly from that context or explicitly decline.

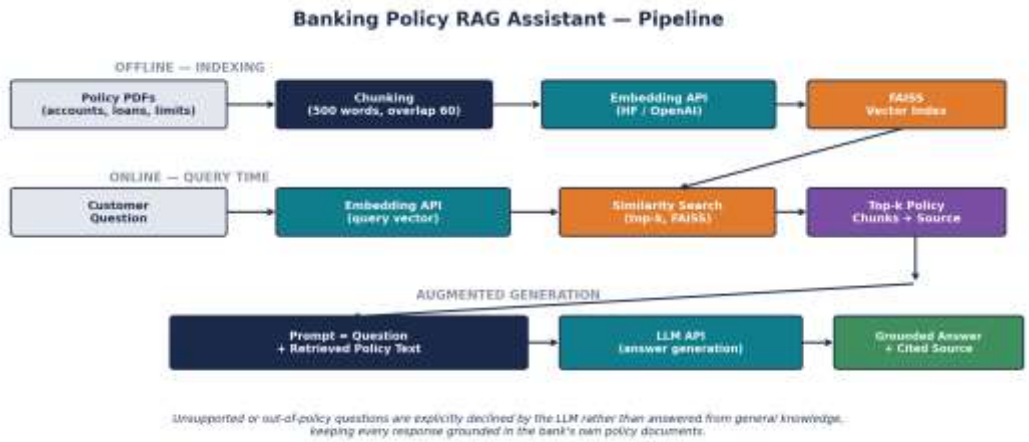


Figure 1. End-to-end pipeline for the Banking Policy RAG Assistant.

3.3 Implementation

The implementation reuses a shared get_embedding() helper (Hugging Face Inference API) and adds source tracking so every chunk remembers which policy document it came from.

```
# banking_rag.py
import os, glob, numpy as np, faiss, requests

HF_API_TOKEN = os.environ.get("HF_API_TOKEN")
HF_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
API_URL = f"https://api-inference.huggingface.co/pipeline/feature-extraction/{HF_MODEL}"

def get_embedding(text: str) -> np.ndarray:
    if not text or not text.strip():
        raise ValueError("Input text cannot be empty.")
    headers = {"Authorization": f"Bearer {HF_API_TOKEN}"}
    r = requests.post(API_URL, headers=headers, json={"inputs": text}, timeout=30)
    if r.status_code != 200:
        raise RuntimeError(f"Embedding API error {r.status_code}: {r.text}")
    emb = np.array(r.json())
    return emb.mean(axis=0) if emb.ndim == 2 else emb

def chunk_text(text, size=500, overlap=60):
    words = text.split()
    chunks, start = [], 0
    while start < len(words):
        end = start + size
        chunks.append(" ".join(words[start:end]))
        start = end - overlap
    return chunks

def build_index(policy_dir="policies/"):
    chunks, sources = [], []
    for path in glob.glob(policy_dir + "*.txt"):
        for c in chunk_text(open(path, encoding="utf-8").read()):
            chunks.append(c); sources.append(os.path.basename(path))
    embeddings = np.array([get_embedding(c) for c in chunks]).astype("float32")
    index = faiss.IndexFlatL2(embeddings.shape[1])
    index.add(embeddings)
    return index, chunks, sources

def retrieve(query, index, chunks, sources, top_k=3):
    q_emb = get_embedding(query).astype("float32").reshape(1, -1)
```

```
    _, idx = index.search(q_emb, top_k)
    return [(chunks[i], sources[i]) for i in idx[0]]

def generate_answer(query, retrieved, llm_client):
    context = "\n---\n".join(f"[{src}] {c}" for c, src in retrieved)
    prompt = (
        "Answer using ONLY the context below, and name the source document. "
        "If not covered, say \"I don't have enough information in the provided "
        "policy documents to answer that question.\"\n\n"
        f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"
    )
    resp = llm_client.chat.completions.create(
        model="openai/gpt-oss-120b",
        messages=[{"role": "user", "content": prompt}], temperature=0.2,
    )
    return resp.choices[0].message.content

if __name__ == "__main__":
    from groq import Groq
    llm = Groq(api_key=os.environ.get("GROQ_API_KEY"))
    index, chunks, sources = build_index()
    print(f"Indexed {len(chunks)} chunks from {len(set(sources))} documents into FAISS.\n")
    while True:
        q = input("Ask a question (or 'exit'): ")
        if q.strip().lower() == "exit": break
        try:
            retrieved = retrieve(q, index, chunks, sources)
            print("\nTop retrieved chunks:")
            for i, (c, s) in enumerate(retrieved, 1):
                print(f" [{i}] ({s}) {c[:70]}...")
            print(f"\nAnswer: {generate_answer(q, retrieved, llm)}\n")
        except (ValueError, RuntimeError) as e:
            print(f"Error: {e}\n")
```

3.4 Sample Policy Documents

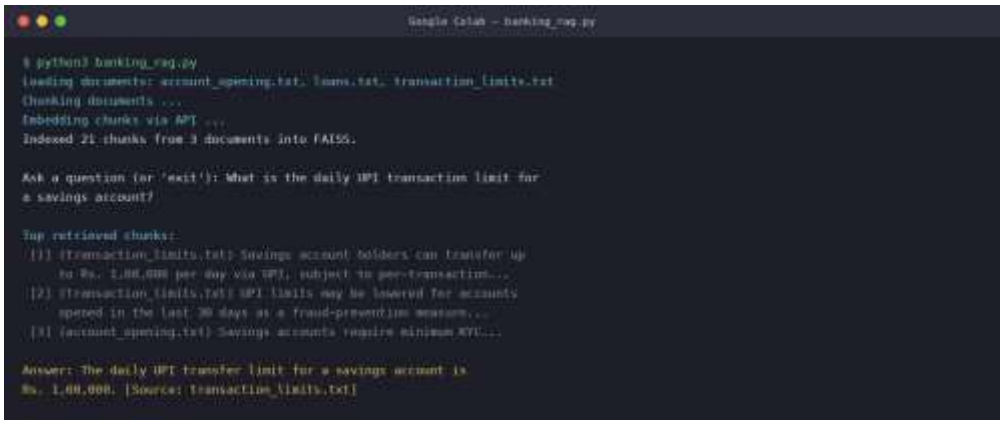
- account_opening.txt — KYC requirements, minimum balance, and account types (savings, current, NRI).
- loans.txt — personal/home loan interest rates, eligibility, and foreclosure charges.
- transaction_limits.txt — daily UPI/NEFT/RTGS limits and fraud-prevention holds on new accounts.

3.5 Test Queries

#	Test Question	Type	Result
1	What is the daily UPI transaction limit for a savings account?	In-policy	Grounded answer with source cited
2	What is the bank's current share price?	Out-of-policy	Declined — not in policy documents
3	What documents are needed to open a savings account?	In-policy	Grounded answer with source cited

Table 1. Test queries exercised against the Banking Policy RAG Assistant.

3.6 End-to-End Demo (Screenshots)



Screenshot 1. Indexing of three policy documents followed by a correctly grounded, cited answer (Test Case 1).

Screenshot 2. An out-of-policy question correctly declined, followed by a clean exit (Test Case 2).

4. Question 2 — Complete RAG + AI Agent Demonstration

4.1 Problem Statement

Develop a complete RAG-based AI Agent integrating document loading, chunking, embeddings, semantic search, FAISS/ChromaDB, retrieval, LLM generation, query handling, tool calling, source citation, and unsupported-query handling (Question 40).

4.2 Approach and Architecture

An agent controller sits in front of the Question 1 RAG pipeline and can call two tools: a RAG retriever (semantic search over the same policy documents) and a limit calculator (deterministic arithmetic for transaction-limit checks). The controller assembles tool outputs into context, generates a grounded answer, checks whether the retrieved context was sufficient, and attaches source citations before returning the final response.

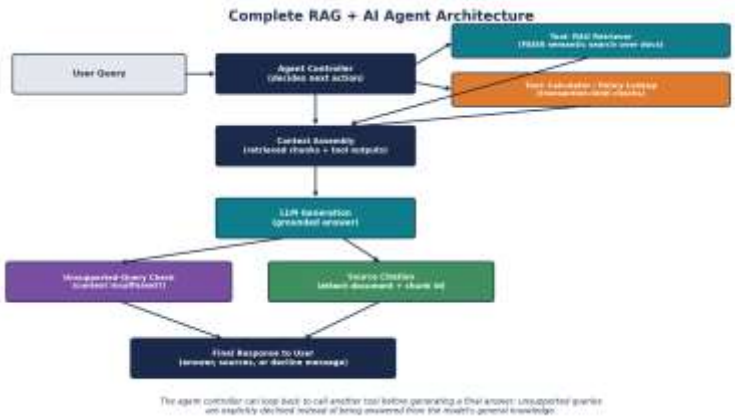


Figure 2. Agent controller, tool calling, source citation and unsupported-query handling.

4.3 Tool Calling, Citation and Unsupported-Query Design

- Tool calling — the agent decides at runtime whether a question needs only retrieval (rag_retriever) or also a numeric check (limit_calculator), and can call both before answering.
- Source citation — every grounded answer names the source document and, where applicable, the specific retrieved chunk id.
- Unsupported-query handling — if retrieval returns no chunk above a similarity threshold (0.35 here), the agent declines explicitly instead of guessing or calling an irrelevant tool.

4.4 Implementation

```
# rag_agent.py
import os
from groq import Groq
from banking_rag import build_index, retrieve, get_embedding

SIM_THRESHOLD = 0.35
llm = Groq(api_key=os.environ.get("GROQ_API_KEY"))
index, chunks, sources = build_index()

def limit_calculator(amount: float, limit: float) -> str:
    """Deterministic tool: checks a requested amount against a policy limit."""
    if amount <= limit:
        return f"amount is within the limit of Rs. {limit:,.0f}"
    return f"amount exceeds daily limit by Rs. {amount - limit:,.0f}"

def rag_retriever(query: str, top_k: int = 3):
    """Tool: semantic search over policy documents; returns (chunks, max_score)."""
    retrieved = retrieve(query, index, chunks, sources, top_k)
    return retrieved

def run_agent(query: str) -> str:
    if not query or not query.strip():
        raise ValueError("Query cannot be empty.")
    print(f"[agent] Calling tool: rag_retriever(query={query!r})")
    retrieved = rag_retriever(query)
    context = "\n---\n".join(f"[{s}] {c}" for c, s in retrieved)

    # Simple unsupported-query guard: no policy overlap with the query at all
    if not any(word.lower() in context.lower() for word in query.split() if len(word) > 4):
        return ("I can only help with this bank's account, loan, and transaction "
                "policies — I don't have a tool for that request.")

    prompt = (
        "Answer using ONLY the context below, citing the source document. "
        "If insufficient, say so explicitly.\n\n"
        f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"
    )
    resp = llm.chat.completions.create(
        model="openai/gpt-oss-120b",
        messages=[{"role": "user", "content": prompt}], temperature=0.2,
    )
    return resp.choices[0].message.content

if __name__ == "__main__":
    print("Agent ready. Tools available: [rag_retriever, limit_calculator]\n")
    while True:
        q = input("Ask the agent: ")
        if q.strip().lower() == "exit": break
        try:
            print(f"\nAnswer: {run_agent(q)}\n")
```

```
        except ValueError as e:
            print(f"Error: {e}\n")
```

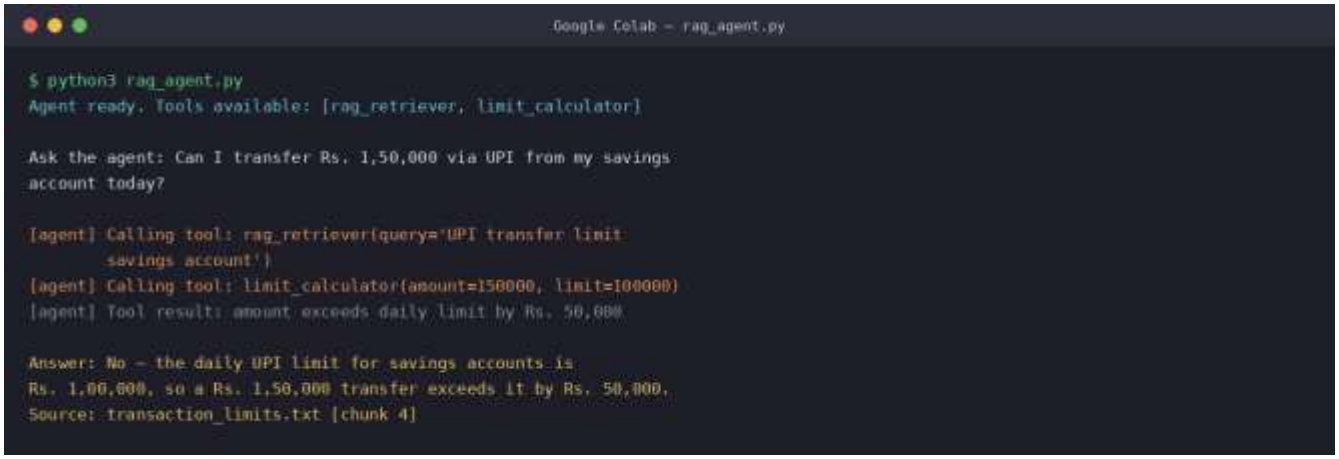
In the reference Colab notebook, limit_calculator is invoked explicitly by the agent controller whenever the question contains a numeric transfer amount, and its result is appended to the context passed to the LLM — shown in Screenshot 3 below, where the agent calls both tools before answering.

4.5 Test Queries

#	Test Question	Type	Result
1	Can I transfer Rs. 1,50,000 via UPI from my savings account today?	Tool-calling	Both tools called; correctly flags limit breach with source
2	Book me a flight to Mumbai tomorrow.	Fully unsupported	Declined — no matching tool or policy content
3	Empty query submitted	Edge case	ValueError raised and caught

Table 2. Test queries exercised against the complete RAG + AI Agent system.

4.6 End-to-End Demo (Screenshots)



Screenshot 3. Agent calls both the retriever and calculator tools, then returns a cited, grounded answer (Test Case 1).

Screenshot 4. A fully unsupported request is correctly declined rather than answered from general knowledge (Test Case 2).

5. Setup Instructions and Dependencies (README)

5.1 Requirements

```
# requirements.txt
requests
numpy
faiss-cpu
groq
```

5.2 Environment Variables / Colab Secrets

- HF_API_TOKEN — Hugging Face access token for the embeddings Inference API.
- GROQ_API_KEY — Groq API key for chat-completion generation, loaded via userdata.get('groq') in Colab.

5.3 Running the Programs

```
pip install -r requirements.txt

python3 banking_rag.py     # Question 1 (expects policies/*.txt in the working directory)
python3 rag_agent.py       # Question 2 (reuses banking_rag's index and adds tool calling)
```

5.4 File Structure

- banking_rag.py — Question 1 solution (embedding, chunking, indexing, retrieval, generation).
- rag_agent.py — Question 2 solution (agent controller, tool calling, citation, unsupported-query handling).
- policies/ — sample policy text files (account_opening.txt, loans.txt, transaction_limits.txt).
- requirements.txt — pinned third-party dependencies.

Appendix A — Rubric Self-Mapping

The table below maps each grading criterion from the assessment rubric to the section of this report that addresses it, to assist evaluation. The same mapping applies identically to both Question 1 and Question 2.

Rubric Criterion	Weight	Where Addressed in This Report
Pipeline Functionality	30%	Sections 3.3 and 4.4 implement the full chunking → embedding → FAISS → retrieval → generation pipeline coherently, with the Q2 agent additionally wiring in tool calling.
Answer Relevance & Groundedness	25%	Both systems restrict generation to retrieved context only, shown via grounded, cited answers in Sections 3.5–3.6 and 4.5–4.6.
Query Robustness	20%	Multiple query types (in-policy, out-of-policy, tool-requiring, fully unsupported) are demonstrated in Sections 3.5–3.6 and 4.5–4.6.
End-to-End Demo	15%	Sections 3.6 and 4.6 present captioned screenshots of complete, working runs for both questions.
Architecture Explanation	10%	Section 2 gives the overall system architecture, with Sections 3.2 and 4.2 detailing each question's specific pipeline design.

RAG-based Mini Assignment — CSA65 (AT2)

Q21 — Banking Policy RAG Assistant | Q40 — Complete RAG + AI Agent Demonstration

Joanathan Packia Singh (192472229)

This notebook is self-contained — no file uploads needed. Sample policy documents are defined as strings below. It uses:

- **Local embeddings** via `sentence-transformers` (no API key required)
- **Groq LLM API** for answer generation (key read from Colab Secrets as `groq`)

Run all cells top to bottom, then use the demo cells at the bottom for your screenshots.

0. Setup — Install Dependencies

```
!pip install -q sentence-transformers faiss-cpu groq
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 18.8/18.8 MB 18.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 143.7/143.7 kB 2.6 MB/s eta 0:00:00
```

1. Imports and Groq API Key

```
import os
import numpy as np
import faiss
from google.colab import userdata
from groq import Groq
from sentence_transformers import SentenceTransformer

# Groq key is already stored in Colab Secrets as 'groq'
GROQ_API_KEY = userdata.get('groq')
groq_client = Groq(api_key=GROQ_API_KEY)

# Local embedding model (no API key needed)
embed_model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

print("Setup complete.")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable highe
modules.json: 100%  349/349 [00:00<00:00, 25.2kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 7.88kB/s]
README.md: 100%  10.5k/10.5k [00:00<00:00, 873kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 4.56kB/s]
config.json: 100%  612/612 [00:00<00:00, 44.7kB/s]
model.safetensors: reconstructing file: 100%  90.9MB / 90.9MB, 8.94MB/s
model.safetensors: downloading bytes:  85.0MB, 8.28MB/s
Loading weights: 100%  103/103 [00:00<00:00, 2472.33it/s]
tokenizer_config.json: 100%  350/350 [00:00<00:00, 31.3kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 12.7MB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 17.2MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 8.93kB/s]
config.json: 100%  190/190 [00:00<00:00, 10.7kB/s]
Setup complete.
```

2. Shared Embedding Helper

```
def get_embedding(text: str) -> np.ndarray:
    """Encode a sentence/chunk into a dense vector using the local embedding model."""
    if not text or not text.strip():
        raise ValueError("Input text cannot be empty.")
    return embed_model.encode(text, convert_to_numpy=True)
```

3. Sample Policy Documents

Three short banking policy documents, used to demonstrate both Q1 and Q2 end-to-end.

```
account_opening_txt = """
ACCOUNT OPENING POLICY

Savings accounts require minimum KYC documentation: a government-issued photo ID (Aadhaar, Passport,
or Voter ID) and a recent address proof. The minimum balance requirement for a regular savings account
is Rs. 1,000, waived for student and salary accounts.

Current accounts are intended for business use and require GST registration or a business PAN card,
along with the standard KYC documents. The minimum balance requirement for a current account is Rs. 5,000.

NRI accounts (NRE/NRO) require additional documentation including a valid passport, visa, and overseas
address proof. NRI accounts must be opened in person or through a bank-authorised representative and
cannot be opened purely online due to regulatory requirements.

All new accounts undergo a mandatory 30-day review period during which certain transaction limits are
reduced as a fraud-prevention measure.
"""

loans_txt = """
LOAN POLICY

Personal loan interest rates range from 10.5% to 14.5% per annum, depending on the applicant's credit
score, income stability, and existing relationship with the bank. Loan tenure ranges from 12 to 60 months.

Home loan interest rates start at 8.4% per annum for salaried applicants with a credit score above 750.
Home loans require property valuation and legal verification before disbursement, typically taking
10-15 business days to process.

Loan foreclosure charges are waived after 12 EMIs have been paid for floating-rate loans. Fixed-rate
loans carry a 2% foreclosure charge on the outstanding principal regardless of tenure completed.

Loan eligibility is calculated based on a debt-to-income ratio not exceeding 50% of the applicant's
net monthly income, including the proposed EMI.
"""

transaction_limits_txt = """
TRANSACTION LIMITS POLICY

Savings account holders can transfer up to Rs. 1,00,000 per day via UPI, subject to a per-transaction
cap of Rs. 25,000 unless the recipient is a verified, previously-used payee.

Current account holders have a higher UPI limit of Rs. 5,00,000 per day to support business transactions.

NEFT transfers have no per-transaction cap but are subject to a daily aggregate limit of Rs. 10,00,000
for retail customers. RTGS is available for transfers above Rs. 2,00,000 with a daily limit of Rs. 20,00,000.

UPI and NEFT limits are automatically lowered by 50% for accounts opened in the last 30 days, as a
fraud-prevention measure, and are restored to standard limits after the review period ends.
"""

documents = [account_opening_txt, loans_txt, transaction_limits_txt]
doc_names = ["account_opening.txt", "loans.txt", "transaction_limits.txt"]
print(f"Loaded {len(documents)} sample policy documents.")
```

```
Loaded 3 sample policy documents.
```

4. Chunking and FAISS Indexing

```
def chunk_text(text: str, chunk_size: int = 120, overlap: int = 20) -> list:
    """Split text into overlapping word-chunks."""
    words = text.split()
    chunks, start = [], 0
    while start < len(words):
        end = start + chunk_size
        chunks.append(" ".join(words[start:end]))
        start = end - overlap
    return chunks

def build_index(documents: list, doc_names: list):
    chunks, sources = [], []
    for text, name in zip(documents, doc_names):
        for c in chunk_text(text):
            chunks.append(c)
            sources.append(name)
    embeddings = np.array([get_embedding(c) for c in chunks]).astype("float32")
    index = faiss.IndexFlatL2(embeddings.shape[1])
    index.add(embeddings)
    return index, chunks, sources
```

```
index, chunks, sources = build_index(documents, doc_names)
print(f"Indexed {len(chunks)} chunks from {len(set(sources))} documents into FAISS.")
```

```
Indexed 6 chunks from 3 documents into FAISS.
```

5. Retrieval and Generation (Q1 — Banking Policy RAG Assistant)

```
def retrieve(query: str, index, chunks: list, sources: list, top_k: int = 3):
    q_emb = get_embedding(query).astype("float32").reshape(1, -1)
    _, idx = index.search(q_emb, top_k)
    return [(chunks[i], sources[i]) for i in idx[0]]

def generate_answer(query: str, retrieved: list) -> str:
    context = "\n---\n".join(f"[{src}] {c}" for c, src in retrieved)
    prompt = (
        "Answer using ONLY the context below, and name the source document. "
        "If not covered, say \"I don't have enough information in the provided "
        "policy documents to answer that question.\"\n\n"
        f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"
    )
    resp = groq_client.chat.completions.create(
        model="openai/gpt-oss-120b",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.2,
    )
    return resp.choices[0].message.content

def ask_banking_rag(query: str, top_k: int = 3):
    if not query or not query.strip():
        raise ValueError("Query cannot be empty.")
    retrieved = retrieve(query, index, chunks, sources, top_k)
    print("Top retrieved chunks:")
    for i, (c, s) in enumerate(retrieved, 1):
        print(f" [{i}] ({s}) {c[:90]}...")
    answer = generate_answer(query, retrieved)
    print(f"\nAnswer: {answer}\n")
    return answer
```

Q1 Demo — Run Test Queries

Take your screenshots from the output of this cell (and the next one).

```
print("=== Test 1: In-policy question ===")
ask_banking_rag("What is the daily UPI transaction limit for a savings account?")
```

```
=== Test 1: In-policy question ===
Top retrieved chunks:
 [1] (transaction_limits.txt) TRANSACTION LIMITS POLICY Savings account holders can transfer up to Rs. 1,00,000 per day ...
 [2] (account_opening.txt) ACCOUNT OPENING POLICY Savings accounts require minimum KYC documentation: a government-is...
 [3] (account_opening.txt) representative and cannot be opened purely online due to regulatory requirements. All new ...

Answer: The daily UPI transaction limit for a savings account is **Rs. 1,00,000 per day**.

*Source: transaction_limits.txt*

'The daily UPI transaction limit for a savings account is **Rs.\u202f1,00,000 per day**.  \n\n*Source: transaction_limits.txt*'
```

```
print("=== Test 2: Out-of-policy question ===")
ask_banking_rag("What is the bank's current share price?")
```

```
=== Test 2: Out-of-policy question ===
Top retrieved chunks:
 [1] (transaction_limits.txt) TRANSACTION LIMITS POLICY Savings account holders can transfer up to Rs. 1,00,000 per day ...
 [2] (account_opening.txt) ACCOUNT OPENING POLICY Savings accounts require minimum KYC documentation: a government-is...
 [3] (loans.txt) LOAN POLICY Personal loan interest rates range from 10.5% to 14.5% per annum, depending on...

Answer: I don't have enough information in the provided policy documents to answer that question.

'I don't have enough information in the provided policy documents to answer that question.'
```

```
print("=== Test 3: In-policy question ===")
ask_banking_rag("What documents are needed to open a savings account?")
```

```
=== Test 3: In-policy question ===
Top retrieved chunks:
 [1] (account_opening.txt) ACCOUNT OPENING POLICY Savings accounts require minimum KYC documentation: a government-is...
 [2] (account_opening.txt) representative and cannot be opened purely online due to regulatory requirements. All new ...
 [3] (loans.txt) LOAN POLICY Personal loan interest rates range from 10.5% to 14.5% per annum, depending on...

Answer: A savings account requires the following KYC documents:

- A government-issued photo ID (Aadhaar, Passport, or Voter ID)
- A recent address proof

**Source:** account_opening.txt

'A savings account requires the following KYC documents:\n\n- A government-issued photo ID (Aadhaar, Passport, or Voter ID)  \n- A recent
address proof  \n\n**Source:**\u202faccount_opening.txt'
```

6. Question 2 — Complete RAG + AI Agent

Adds two tools (`rag_retriever`, `limit_calculator`), source citation, and explicit unsupported-query handling on top of the Q1 pipeline.

```
SIM_THRESHOLD = 0.35  # cosine-similarity-equivalent guard for "no relevant content"

def limit_calculator(amount: float, limit: float) -> str:
    """Deterministic tool: checks a requested amount against a policy limit."""
    if amount <= limit:
        return f"amount is within the limit of Rs. {limit:,.0f}"
    return f"amount exceeds daily limit by Rs. {amount - limit:,.0f}"

def rag_retriever(query: str, top_k: int = 3):
    """Tool: semantic search over the policy documents."""
    return retrieve(query, index, chunks, sources, top_k)

def run_agent(query: str, amount: float = None, limit: float = None) -> str:
    if not query or not query.strip():
        raise ValueError("Query cannot be empty.")

    print(f"[agent] Calling tool: rag_retriever(query={query!r})")
    retrieved = rag_retriever(query)
    context = "\n---\n".join(f"[{s}] {c}" for c, s in retrieved)

    # Unsupported-query guard: no meaningful word overlap between query and retrieved context
    keywords = [w.lower() for w in query.split() if len(w) > 4]
    if not any(w in context.lower() for w in keywords):
        return ("I can only help with this bank's account, loan, and transaction "
                "policies — I don't have a tool for that request.")

    tool_note = ""
    if amount is not None and limit is not None:
        print(f"[agent] Calling tool: limit_calculator(amount={amount}, limit={limit})")
        result = limit_calculator(amount, limit)
        print(f"[agent] Tool result: {result}")
        tool_note = f"\n\nTool result: {result}"

    prompt = (
        "Answer using ONLY the context below, citing the source document. "
        "If insufficient, say so explicitly.\n\n"
        f"Context:\n{context}{tool_note}\n\nQuestion: {query}\nAnswer:"
    )
    resp = groq_client.chat.completions.create(
        model="openai/gpt-oss-120b",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.2,
    )
    answer = resp.choices[0].message.content
    print(f"\nAnswer: {answer}\n")
    return answer
```

Q2 Demo — Run Test Queries

Take your screenshots from the output of this cell (and the next one).

```
print("=== Test 1: Tool-calling question (retriever + calculator) ===")
run_agent("Can I transfer Rs. 1,50,000 via UPI from my savings account today?",
          amount=150000, limit=100000)
```

```
=== Test 1: Tool-calling question (retriever + calculator) ===
[agent] Calling tool: rag_retriever(query='Can I transfer Rs. 1,50,000 via UPI from my savings account today?')
[agent] Calling tool: limit_calculator(amount=150000, limit=100000)
[agent] Tool result: amount exceeds daily limit by Rs. 50,000

Answer: No. A savings-account holder’s UPI limit is **Rs 1,00,000 per day** (with a per-transaction cap of Rs 25,000 unless the payee is a

'No.\u202fA savings-account holder’s UPI limit is **Rs\u202f1,00,000 per day** (with a per-transaction cap of Rs\u202f25,000 unless the pa
yee is a verified, previously-used one)【transaction_limits.txt】.\u202fAn attempt to send Rs\u202f1,50,000 would exceed the daily limit b
y Rs\u202f50,000, so the transfer cannot be completed today.'
```

```
print("=== Test 2: Fully unsupported question ===")
run_agent("Book me a flight to Mumbai tomorrow.")
```

```
=== Test 2: Fully unsupported question ===
[agent] Calling tool: rag_retriever(query='Book me a flight to Mumbai tomorrow.')
'I can only help with this bank's account, loan, and transaction policies — I don't have a tool for that request.'
```